

5. Asynchronous gRPC: Synchronous vs. Asynchronous Execution

In the Section 4, the synchronous server gives every running request its own worker thread and when a handler needs to wait for disk, database or another service, its thread is blocked and cannot do anything. However, the asynchronous version uses Python's *asyncio* and *grpc.aio*. All requests run as co-routines on one event loop, and the control is handed back to the loop whenever a co-routine must wait by the keyword *await*. This way other requests can run in the meantime. The contract *rental.proto* and OOP classes are not changed since both servers implement the same *RentalService*.

What changed from the synchronous server

Aspect	Synchronous (Section 4)	Asynchronous (this section)
Server object	<code>grpc.server(ThreadPoolExecutor(10))</code>	<code>grpc.aio.server()</code>
Handlers	<code>def RentBook(self, request, context)</code>	<code>async def RentBook(self, request, context)</code>
Artificial delay	<code>time.sleep(delay)</code> blocks one thread	<code>await asyncio.sleep(delay)</code> yields to the loop
Concurrency	one thread per running request, at most 10 at once	one thread, one event loop, any number of waiting requests
Client	<code>grpc.insecure_channel</code> , blocking stub calls	<code>grpc.aio.insecure_channel</code> , <code>await stub.Method(...)</code>
Port	<code>localhost:50051</code>	<code>localhost:50052</code>
OOP code	<code>RentalManager</code> , <code>BookCatalog</code> (unchanged)	the same classes (unchanged)

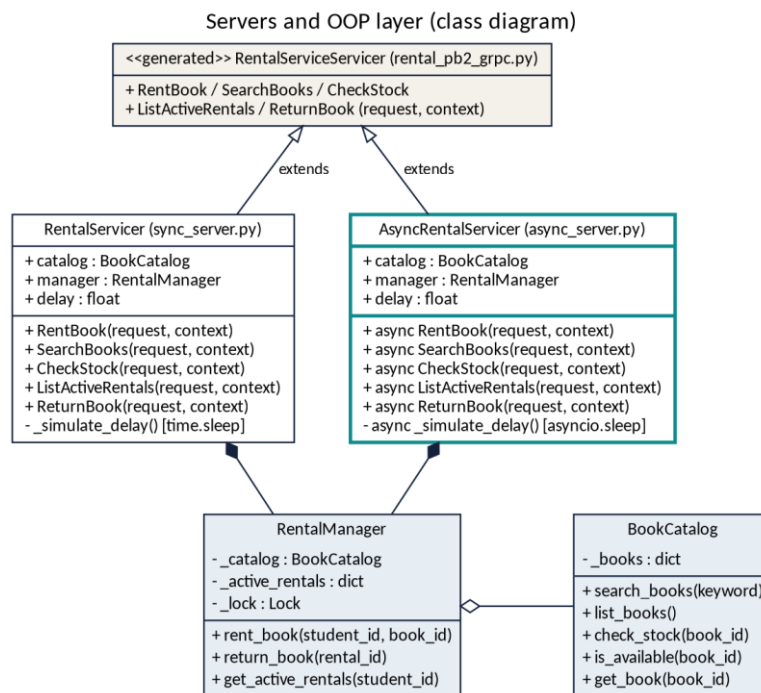


Figure 1. UML class diagram. Both servicers extend the generated base class and own one `RentalManager`; only the handlers differ.

The asynchronous server

`async_server.py` defines `AsyncRentalServicer`. It creates the catalog and the manager once, so stock and rentals are persistent between calls. Just like in the synchronous server. The conversion helpers like `book_to_pb`, `rental_to_pb` and `result_to_pb` are imported from `sync_server.py`. They are not copied. In this case, the handler still only reads the request, calls the OOP layer and prints the result into a message:

```
async def _simulate_delay(self):
    if self.delay > 0:
        await asyncio.sleep(self.delay)      # yields to the event loop

async def SearchBooks(self, request, context):
    await self._simulate_delay()
    if request.keyword:
        books = self.catalog.search_books(request.keyword)
    else:
        books = self.catalog.list_books()
    return rental_pb2.SearchResponse(books=[book_to_pb(b) for b in books])
```

- **No await on the OOP calls.** To make the handlers call them directly `search_books`, `rent_book` and the others are plain, fast functions with no I/O. On the other hand, the lock inside `RentalManager` is held for a few microseconds and never across an `await`. This way, it cannot cause a deadlock.
- **Never time.sleep.** `--delay` option uses `asyncio.sleep` only to prevent the freezing of whole event loop by `time.sleep` inside the `async def`. If this was not implemented, when `time.sleep` freeze the event loop, every other request would wait with it and there would be no concurrency.
- **Server start-up.** `grpc.aio.server()`, `add_insecure_port("localhost:50052")`, `await server.start()` and `await server.wait_for_termination()`, run by `asyncio.run`. The helper `create_server(port=0)` lets the tests use a free port.

The asynchronous client

There are 2 demos of `async_client.py`. While Demo 1 makes simple awaited calls like `rent`, `out of stock`, `unknown book`, Demo 2 does the important part: ten students search at the same time. In this case, `asyncio.gather` starts all the ten co-routines and each one sends its request immediately and is suspended at its `await`, after that it resumes when its reply arrives:

```
results = await asyncio.gather(
    *(one_search(stub, i + 1, kw, timeout, t0)
      for i, kw in enumerate(SEARCH_KEYWORDS))
)
```

While the searches are still ongoing, there is a small background task which prints “still working” in every 0.1 s to show the client is not frozen. Every call has a `timeout=`, and errors are caught as `grpc.aio.AioRpcError`. A stopped server gives an error message like `UNAVAILABLE`, and the slow server gives the message `DEADLINE_EXCEEDED`. Same with the messages in `sync_client.py`.

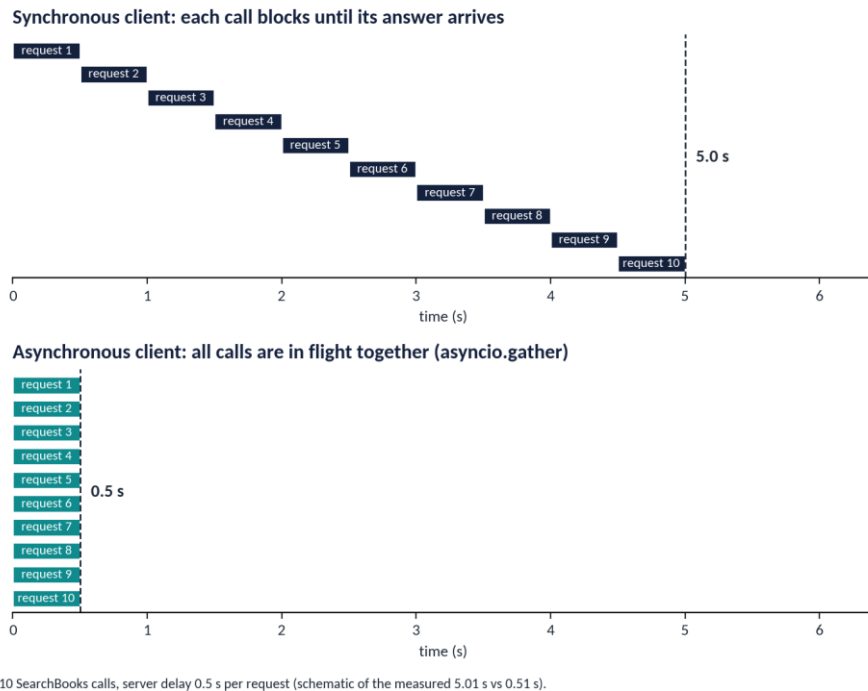


Figure 2. Ten searches with a 0.5 s server delay: one after another (top) and with `asyncio.gather` (bottom).

Measured results

`compare_sync_async.py` starts both servers in one process with the same delay of 0.5 s per request and times the same SearchBooks calls. Rows (a) and (b) are the main comparisons. Rows (c) and (d) show that a thread-pool server can also run requests in parallel, within the limits of its pool.

Scenario	Requests	Time
(a) sync client, one call after another	10	5.01 s
(b) async client, <code>asyncio.gather</code>	10	0.51 s
(c) sync server (10 workers), threaded clients	50	2.52 s
(d) async server, <code>asyncio.gather</code> (one thread)	50	0.51 s

Row (a) is about N times the delay (10×0.5 s) and row (b) is about one delay, a speed-up of 9.9 times. In row (c), 50 requests on 10 workers need five rounds of 0.5 s, which gives 2.5 s. The async server in row (d) is not limited by a pool, so 50 requests take as long as 10. Each number is one run on one machine over localhost, and the times vary by a few hundredths of a second between runs.

When async helps, and when it does not

Asynchronous handling helps when requests spend their time waiting. The delay in our experiment is simulated; the real OOP methods finish in microseconds, so with `--delay 0` the two servers feel the same on this project. When a handler waits for a database, a file or another network call, the gain appears. Async is not used to speed up CPU-heavy work: a long computation inside an `async def` handler blocks the event loop and delays every other request. As row (c) shows, a thread pool also serves parallel clients, however it need a thread per running request and is limited by its size.

The OOP layer is shared by both servers and is protected by the existing threading. Lock in RentalManager. `compare_sync_async.py` also sends 10 concurrent `RentBook` calls for B003, which has a single copy, to each server: exactly 1 of 10 succeeds on both, and the other 9 receive `success = False`. A test with 20 concurrent calls gives the same result.

Testing and how to run

`test_async_grpc.py` has 18 tests built on `unittest.IsolatedAsyncioTestCase`. Each test starts a real `grpc.aio` server on a free port and calls it over the network. They cover rentals, out of stock, unknown book, empty student ID, case-insensitive search, stock check, active rentals, return, 10 concurrent searches, and the timing test (ten gathered searches with a 0.3 s delay finish in under 1.5 s instead of 3 s). They also cover the 20 concurrent rentals and `DEADLINE_EXCEEDED`. All 50 tests of the project pass: 13 for the OOP layer, 19 for the synchronous server, and 18 for the asynchronous one.

```
pip install -r requirements.txt
python async_server.py --delay 0.5      # terminal 1
python async_client.py                  # terminal 2
python compare_sync_async.py --delay 0.5 # starts both servers itself
python -m unittest -v                  # all 50 tests
```